

Oracle Joins

Oracle join is used to combine columns from two or more tables based on the values of the related columns.

The related columns are typically the primary key column(s) of the first table and foreign key column(s) of the second table.

Oracle supports

- I. inner join,
- II. Natural join
- III. LEFT OUTER JOIN (LEFT JOIN)
- IV. RIGHT OUTER JOIN (RIGHT JOIN)
- V. full outer join and
- VI. cross join.

Setting up sample tables

create two new tables with the same structure for the demonstration:

```
CREATE TABLE palette_a (  
  id INT PRIMARY KEY,  
  color VARCHAR2 (100) NOT NULL  
);  
  
CREATE TABLE palette_b (  
  id INT PRIMARY KEY,  
  color VARCHAR2 (100) NOT NULL  
);
```

-- insert data for the palette_a

```
INSERT INTO palette_a (id, color) VALUES (1, 'Red');  
  
INSERT INTO palette_a (id, color) VALUES (2, 'Green');  
  
INSERT INTO palette_a (id, color) VALUES (3, 'Blue');  
  
INSERT INTO palette_a (id, color) VALUES (4, 'Purple');
```

-- insert data for the palette_b

```
INSERT INTO palette_b (id, color) VALUES (1, 'Green');  
  
INSERT INTO palette_b (id, color) VALUES (2, 'Red');  
  
INSERT INTO palette_b (id, color) VALUES (3, 'Cyan');  
  
INSERT INTO palette_b (id, color) VALUES (4, 'Brown');
```

The tables have some common colors such as Red and Green. Let's call the **palette_a** the left table and **palette_b** the right table:

ID	COLOR
1	Red
2	Green
3	Blue
4	Purple

ID	COLOR
1	Green
2	Red
3	Cyan
4	Brown

1. Oracle inner join

The following statement joins the left table to the right table using the values in the color column:

```
SELECT
  a.id id_a,
  a.color color_a,
  b.id id_b,
  b.color color_b
FROM
  palette_a a INNER JOIN palette_b b ON a.color = b.color;
```

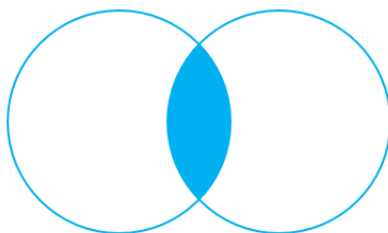
Here is the output:

ID_A	COLOR_A	ID_B	COLOR_B
2	Green	1	Green
1	Red	2	Red

Note: SQL does not guarantee that order in row output. The database optimizer is free to choose an execution plan.

As can be seen clearly from the result, the inner join returns rows from the left table that match the rows from the right table.

The following Venn diagram illustrates an inner join when combining two result sets:



INNER JOIN

2. Oracle left join

The following statement joins the left table with the right table using a **left join** (or a **left outer join**):

```

SELECT
  a.id id_a,
  a.color color_a,
  b.id id_b,
  b.color color_b
FROM
  palette_a a LEFT JOIN palette_b b ON a.color = b.color;

```

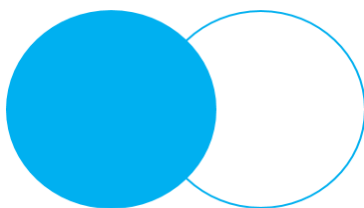
The output is shown as follows:

ID_A	COLOR_A	ID_B	COLOR_B
2	Green	1	Green
1	Red	2	Red
3	Blue	(null)	(null)
4	Purple	(null)	(null)

The left join returns all rows from the left table with the matching rows if available from the right table.

If there is no matching row found from the right table, the left join will have null values for the columns of the right table:

The following Venn diagram illustrates the left join:



LEFT OUTER JOIN

Sometimes, you want to get only rows from the left table that do not exist in the right table. To achieve this, you use the left join and a **WHERE** clause to exclude the rows from the right table.

For example, the following statement shows colors that are only available in the palette_a but not palette_b:

```

SELECT
  a.id id_a,

```

```

a.color color_a,
b.id id_b,
b.color color_b

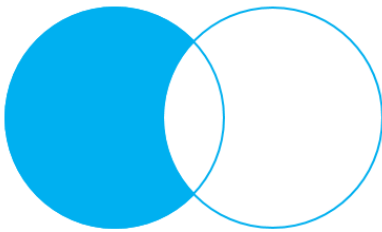
FROM
  palette_a a LEFT JOIN palette_b b ON a.color = b.color
WHERE b.id IS NULL;

```

Here is the output:

ID_A	COLOR_A	ID_B	COLOR_B
3	Blue	(null)	(null)
4	Purple	(null)	(null)

The following Venn diagram illustrates the left join **with** the exclusion of rows from the right table:



LEFT OUTER JOIN – only rows from the left table

3. Oracle right join

The right join or right outer join is a **reversed version of the left join.**

The right join makes a result set that contains all rows from the right table with the matching rows from the left table. If there is **no** match, the left side will have **nulls**.

The following example uses a right join to join the left table to the right table:

```

SELECT
  a.id id_a,
  a.color color_a,
  b.id id_b,
  b.color color_b

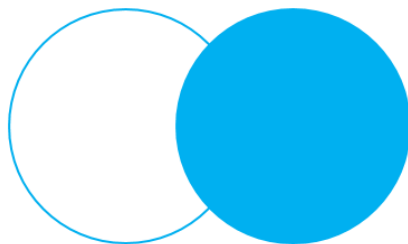
FROM
  palette_a a RIGHT JOIN palette_b b ON a.color = b.color;

```

Here is the output:

ID_A	COLOR_A	ID_B	COLOR_B
1	Red	2	Red
2	Green	1	Green
(null)	(null)	4	Brown
(null)	(null)	3	Cyan

The following Venn diagram illustrates the right join:



RIGHT OUTER JOIN

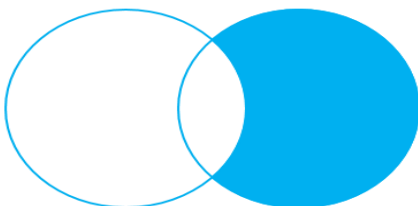
Likewise, you can get only rows from the right table but not the left table by adding a **WHERE** clause to the above statement as shown in the following query:

```
SELECT
  a.id id_a,
  a.color color_a,
  b.id id_b,
  b.color color_b
FROM
  palette_a a RIGHT JOIN palette_b b ON a.color = b.color
WHERE a.id IS NULL;
```

Here is the output:

ID_A	COLOR_A	ID_B	COLOR_B
(null)	(null)	4	Brown
(null)	(null)	3	Cyan

The following Venn diagram illustrates the right join with the exclusion of rows from the left table:



RIGHT OUTER JOIN – only rows from the right table

4. Oracle full outer join

Oracle full outer join or full join returns a result set that contains all rows from both left and right tables, with the matching rows from both sides where available.

If there is no match, the missing side will have nulls.

The following example shows the full outer join of the left and right tables:

```
SELECT
  a.id id_a,
  a.color color_a,
  b.id id_b,
  b.color color_b
FROM
  palette_a a FULL OUTER JOIN palette_b b ON a.color = b.color;
```

The following picture illustrates the result set of the full outer join:

ID_A	COLOR_A	ID_B	COLOR_B
1	Red	2	Red
2	Green	1	Green
3	Blue	(null)	(null)
4	Purple	(null)	(null)
(null)	(null)	4	Brown
(null)	(null)	3	Cyan

Note that the OUTER keyword is optional.

The following Venn diagram illustrates the full outer join:



FULL OUTER JOIN

To get a set of rows that are unique from the left and right tables, you perform the same full join and then exclude the rows that you don't want from both sides using a WHERE clause as follows:

```
SELECT
  a.id id_a,
```

```

a.color color_a,
b.id id_b,
b.color color_b

FROM
  palette_a a FULL JOIN palette_b b ON a.color = b.color

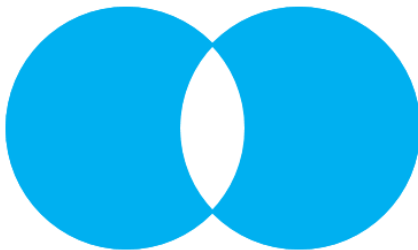
WHERE a.id IS NULL OR b.id IS NULL;

```

Here is the result:

ID_A	COLOR_A	ID_B	COLOR_B
(null)	(null)	3	Cyan
(null)	(null)	4	Brown
3	Blue	(null)	(null)
4	Purple	(null)	(null)

The following Venn diagram illustrates the above operation:



FULL OUTER JOIN – only
rows unique to both tables

5. Oracle CROSS JOIN

In Mathematics, given two sets A and B, the Cartesian product of $A \times B$ is the set of all ordered pair (a,b), which a belongs to A and b belongs to B.

A **cross join** in SQL returns the Cartesian product of two tables, meaning it combines *each row from the first table with every row from the second table*. If you have two tables, palette_a and palette_b, with two columns each (id and color), a **cross join** will create a combination of all rows in palette_a with all rows in palette_b.

Now, suppose these tables have the following data:

palette_a

id	color
1	Red
2	Blue

palette_b

id	color
1	Green
2	Yellow

Cross Join Example:

```
SELECT
    a.id AS id_a,
    a.color AS color_a,
    b.id AS id_b,
    b.color AS color_b
FROM
    palette_a a CROSS JOIN palette_b b;
```

Unlike other joins such as INNER JOIN or LEFT JOIN, CROSS JOIN does not have the ON clause with a join predicate.

Result of the Cross Join:

id_a	color_a	id_b	color_b
1	Red	1	Green
1	Red	2	Yellow
2	Blue	1	Green
2	Blue	2	Yellow

When you perform a cross join of two tables, which have no relationship, you will get a Cartesian product of rows and columns of both tables.

The cross join is useful when you want to generate plenty of rows for testing. Suppose we have two tables that have m and n rows, the Cartesian product of these tables has $m \times n$ rows.

The query combines each row from palette_a with every row from palette_b. Since palette_a has 2 rows and palette_b has 2 rows, the result will have $2 \times 2 = 4$ rows.

This can be useful when you need to generate combinations of items from two different datasets.

6. Natural join

A **NATURAL JOIN** automatically looks for **columns with the same names** in both tables.

In this case, both **id** and **color** exist in **both tables**.

So the join condition becomes:

```
palette_a.id = palette_b.id  
  
AND  
  
palette_a.color = palette_b.color
```

```
SELECT *  
  
FROM palette_a NATURAL JOIN palette_b;
```

This is slightly different from INNER JOIN because **you don't specify the ON condition**.

ID	COLOR
1	Red
2	Green
3	Blue
4	Purple

ID	COLOR
1	Green
2	Red
3	Cyan
4	Brown

Step-by-step check

- For **ID = 1** → a.color = 'Red', b.color = 'Green' → ❌ not equal
- For **ID = 2** → a.color = 'Green', b.color = 'Red' → ❌ not equal
- For **ID = 3** → a.color = 'Blue', b.color = 'Cyan' → ❌ not equal
- For **ID = 4** → a.color = 'Purple', b.color = 'Brown' → ❌ not equal

No rows returned, because there is **no row where both ID and COLOR match**.

Above sql is Equivalent to:

```
SELECT *  
  
FROM palette_a a  
  
JOIN palette_b b  
  
ON a.id = b.id
```

```
AND a.color = b.color;
```

Note : JOIN = shorthand for INNER JOIN